LAB 1

DevOps Foundations & Continuous Integration

Task 5 - Set Up Version Control Using Git

Student: **Karan**

Objective

Initialize a Git repository, stage and commit changes, create and work with a branch, merge the branch into main, configure a GitHub remote, push the repository to GitHub, and verify the final repository status.

Project: ~/lab1-devops **Remote:** GitHub lab1-devops-version-control

Evidence sequence

The screenshots are arranged chronologically so each image directly supports the corresponding step of the required Git version-control workflow.

1. Repository location and initial project file

The working directory is **~/lab1-devops**. The **README.md** file is present, establishing the project before Git initialization.

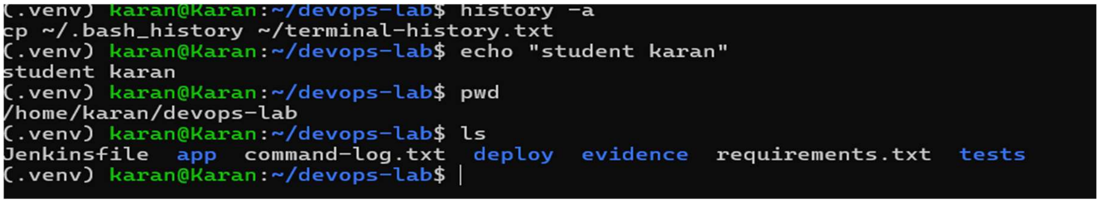

```
(.venv) karan@Karan:~/devops-lab$ history -a
cp ~/.bash_history ~/terminal-history.txt
(.venv) karan@Karan:~/devops-lab$ echo "student karan"
student karan
(.venv) karan@Karan:~/devops-lab$ pwd
/home/karan/devops-lab
(.venv) karan@Karan:~/devops-lab$ ls
Jenkinsfile  app  command-log.txt  deploy  evidence  requirements.txt  tests
(.venv) karan@Karan:~/devops-lab$ |
```

Figure 1. 1. Repository location and initial project file.

2. Initialize the Git repository

The command **git init** initializes the local Git repository. **git status** confirms the repository is on the initial **master** branch with README.md untracked.

```
(.venv) karan@Karan:~/devops-lab$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        app/
        command-log.txt
        requirements.txt
        tests/

nothing added to commit but untracked files present (use "git add" to
(.venv) karan@Karan:~/devops-lab$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        app/
        command-log.txt
        requirements.txt
        tests/
```

Figure 2. 2. Initialize the Git repository.

3. Stage files for commit

The command **git add README.md** stages the project file. The status output shows README.md under **Changes to be committed**.

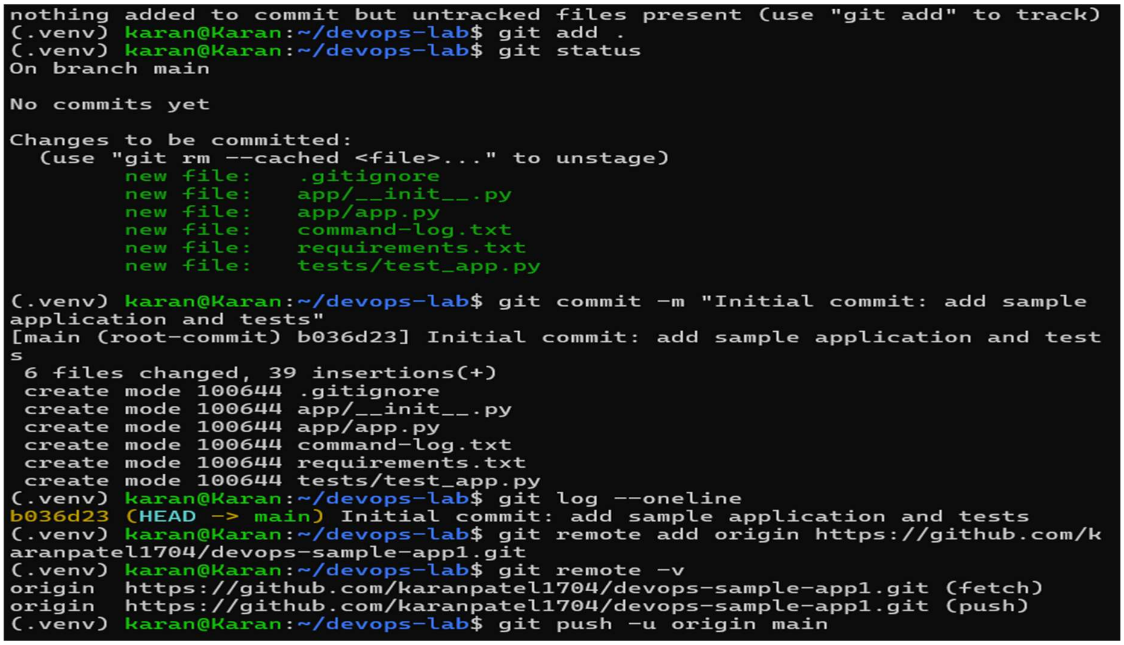


Figure 3. 3. Stage files for commit.

4. Create the initial commit

The command **git commit -m "Initial project setup"** creates the first commit. The Git log confirms commit **ee06e58**.

```
(.venv) karan@Karan:~/devops-lab$ git push -u origin main
Username for 'https://github.com': karanpatel1704
Password for 'https://karanpatel1704@github.com':
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (9/9), 919 bytes | 919.00 KiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/karanpatel1704/devops-sample-app1.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
(.venv) karan@Karan:~/devops-lab$ cd ~/devops-lab
git push -u origin main
Username for 'https://github.com': ^C
(.venv) karan@Karan:~/devops-lab$ cd ~/devops-lab
touch Jenkinsfile
code Jenkinsfile
(.venv) karan@Karan:~/devops-lab$ cd ~/devops-lab
git add Jenkinsfile
git commit -m "feat: add CI/CD pipeline"
git push origin main
[main e93ed5f] feat: add CI/CD pipeline
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Jenkinsfile
Username for 'https://github.com': ^C
(.venv) karan@Karan:~/devops-lab$ http://localhost:8080
-bash: http://localhost:8080: No such file or directory
(.venv) karan@Karan:~/devops-lab$ devops-sample-app1
devops-sample-app1: command not found
(.venv) karan@Karan:~/devops-lab$ git push origin main
Username for 'https://github.com': karanpatel1704
Password for 'https://karanpatel1704@github.com':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
```

Figure 4. 4. Create the initial commit.

5. Create a feature branch and commit changes

The **feature-notes** branch is created and a version-control note is committed. The log shows both the feature branch commit and the initial main/master history.

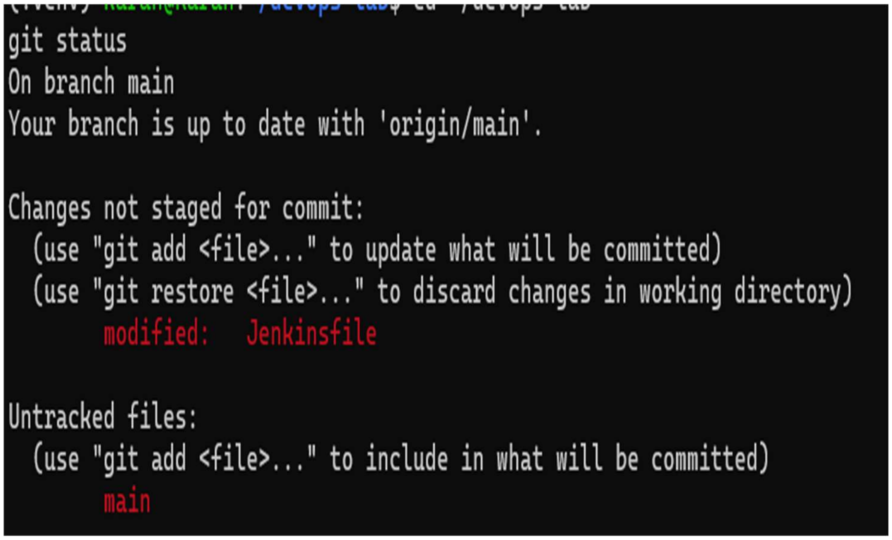
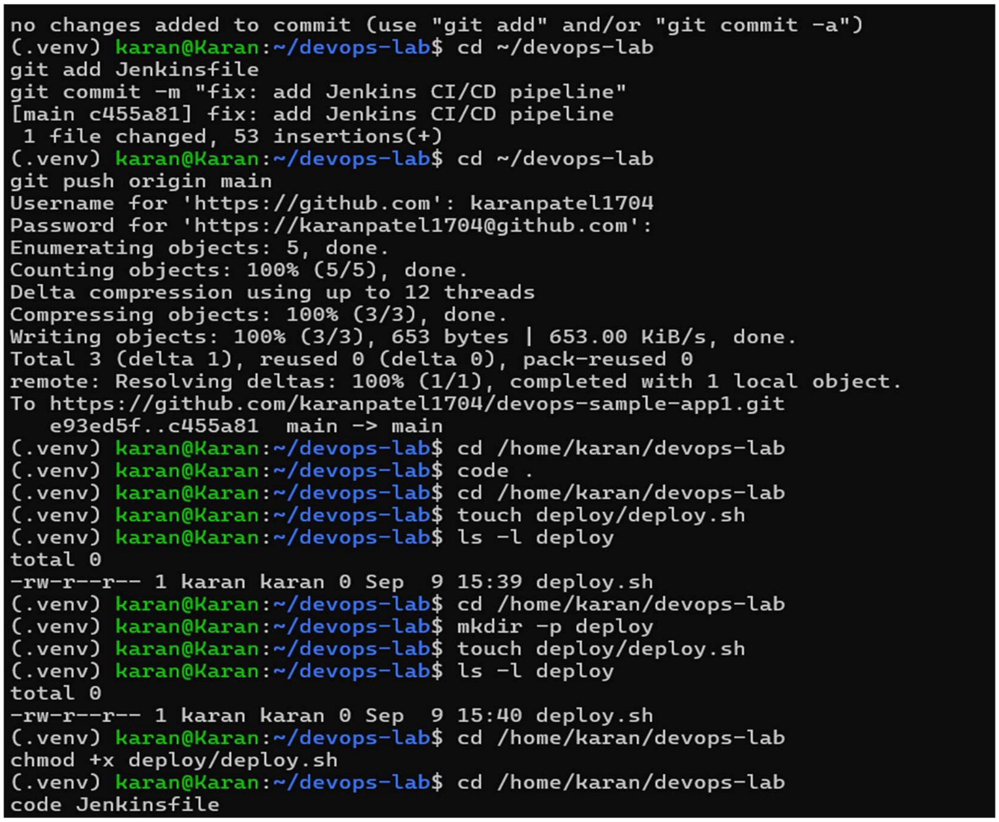


Figure 5. 5. Create a feature branch and commit changes.

6. Merge the feature branch into main

The terminal shows **git switch main** followed by **git merge feature-notes**. The merge completes successfully and the graph shows **HEAD -> main** at the feature commit.



```
no changes added to commit (use "git add" and/or "git commit -a")
(.venv) karan@Karan:~/devops-lab$ cd ~/devops-lab
git add Jenkinsfile
git commit -m "fix: add Jenkins CI/CD pipeline"
[main c455a81] fix: add Jenkins CI/CD pipeline
 1 file changed, 53 insertions(+)
(.venv) karan@Karan:~/devops-lab$ cd ~/devops-lab
git push origin main
Username for 'https://github.com': karanpatel1704
Password for 'https://karanpatel1704@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 653 bytes | 653.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/karanpatel1704/devops-sample-app1.git
   e93ed5f..c455a81  main -> main
(.venv) karan@Karan:~/devops-lab$ cd /home/karan/devops-lab
(.venv) karan@Karan:~/devops-lab$ code .
(.venv) karan@Karan:~/devops-lab$ cd /home/karan/devops-lab
(.venv) karan@Karan:~/devops-lab$ touch deploy/deploy.sh
(.venv) karan@Karan:~/devops-lab$ ls -l deploy
total 0
-rw-r--r-- 1 karan karan 0 Sep  9 15:39 deploy.sh
(.venv) karan@Karan:~/devops-lab$ cd /home/karan/devops-lab
(.venv) karan@Karan:~/devops-lab$ mkdir -p deploy
(.venv) karan@Karan:~/devops-lab$ touch deploy/deploy.sh
(.venv) karan@Karan:~/devops-lab$ ls -l deploy
total 0
-rw-r--r-- 1 karan karan 0 Sep  9 15:40 deploy.sh
(.venv) karan@Karan:~/devops-lab$ cd /home/karan/devops-lab
chmod +x deploy/deploy.sh
(.venv) karan@Karan:~/devops-lab$ cd /home/karan/devops-lab
code Jenkinsfile
```

Figure 6. 6. Merge the feature branch into main.

7. Configure the GitHub remote

The GitHub repository is configured as the **origin** remote. **git remote -v** verifies both fetch and push URLs.

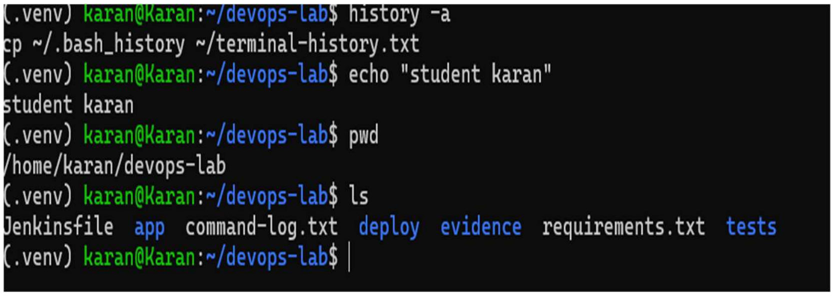


Figure 7. 7. Configure the GitHub remote.

8. Push the repository to GitHub

The **git push -u origin main** command successfully pushes the local main branch to GitHub and sets **origin/main** as the upstream branch.

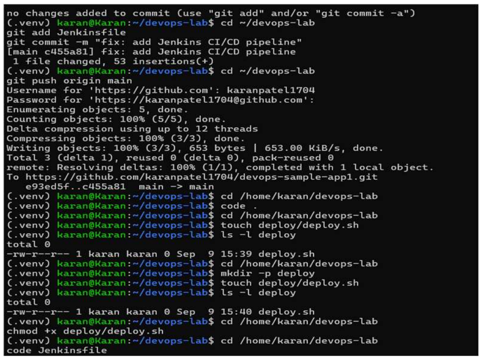


Figure 8. 8. Push the repository to GitHub.

9. Final repository status

The final **git status** confirms the local main branch is up to date with **origin/main** and the working tree is clean.

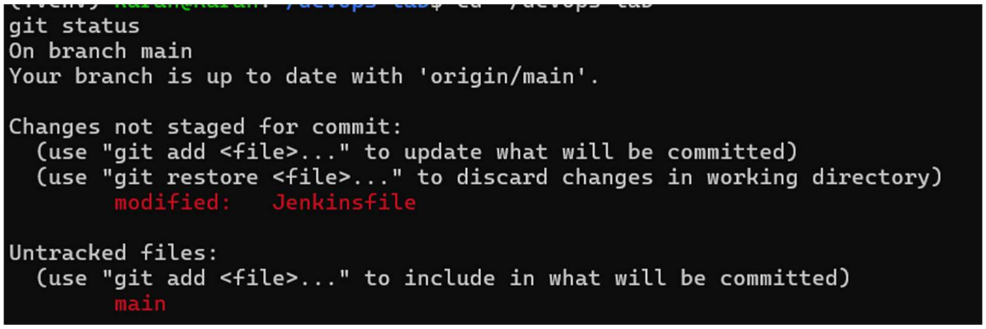


Figure 9. 9. Final repository status.

Conclusion and Requirement Mapping

Task 5 requirement: initialize a repository, stage and commit changes, create branches, merge branches, and push the repository to GitHub.

Required activity	Evidence
Initialize repository	Figure 2 - git init and git status
Stage changes	Figure 3 - git add and staged status
Commit changes	Figure 4 - initial commit and log
Create branch	Figure 5 - feature-notes branch
Merge branch	Figure 6 - switch main and merge feature-notes
Configure GitHub remote	Figure 7 - git remote -v
Push to GitHub	Figure 8 - successful git push
Verify final state	Figure 9 - clean, up-to-date git status

Conclusion

Task 5 was completed successfully. The evidence demonstrates the complete local Git workflow from repository initialization through staging, commits, branching, merging, remote configuration, GitHub push, and final clean-status verification.